

Memory Efficient Doubly Linked List – Delete At Position
–Space Complexity Analysis and Algorithm

Program:

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    // Traverse to the node at 'pos'
    while (curr != nullptr && i < pos)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }
```

```

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    // We are deleting 'curr'.
    // Logic: Connect 'prev' directly to 'next'.

    next = XOR(prev, curr->npx); // Decode node after curr

    // 1. Update prev
    // prev->npx was (prevPrev ^ curr). Needs to be
    (prevPrev ^ next).
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, next);

    // 2. Update next (if exists)
    // next->npx was (curr ^ nextNext). Needs to be (prev ^
    nextNext).
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(prev, nextNext);
    }

    cout << "Deleted " << curr->data << " from position " <<
pos << "." << endl;
    free(curr);
}

.....

    case 7:
        cout << "Enter position to delete: ";
        cin >> pos;
        deleteFromPosition(pos);
        break;

```

Space Complexity Analysis: XOR Linked List (Delete From Position)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameter:** *int pos.*
 - **Analysis:** *The pos parameter is a single integer. It occupies a constant amount of space ($O(1)$) regardless of the size of the list or the value of the position.*
 - **Complexity:** $O(1)$
- **Full Data Definition:**
 - **Function Data:** *The parameter pos + the existing XOR linked list.*
 - **Analysis:** *The function operates on a data structure with n nodes. To perform the deletion, the function must be "aware" of the list context.*
 - **Complexity:** $O(n)$

2. Auxiliary Space Complexity

Definition: This measures the temporary or extra memory used by the algorithm during its execution.

- **Local Pointer Variables:** *The function declares several local pointers: curr, prev, next, prevPrev, and nextNext.*
 - **Analysis:** *These are all stored on the stack. On a 64 – bit system, each takes 8 bytes. The number of these pointers is fixed and does not increase even if the list size (n) increases or if pos is very large.*
- **Local Counter:** *The integer i used for counting nodes during traversal takes constant space.*
- **Traversal:** *The traversal is iterative (using a while loop). It does not use recursion, meaning no extra frames are added to the call stack as the list grows.*
- **XOR Operations:** *The bitwise calculations happen within CPU registers, requiring no extra memory allocation.*
- **Memory Release:** *free(curr) returns memory to the system. While this reduces total memory usage, the auxiliary space used to perform the logic remains constant.*

Conclusion: *The amount of extra memory used is fixed and does not grow as the size of the linked list (n) grows.*

- **Auxiliary Space Complexity: $O(1)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Delete From Position (XOR Doubly Linked List)

XORDELPOS(START, POS):

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete."

Return and EXIT.

[End of If structure]

2. [DELETE FROM BEGINNING?]

If POS = 1, then:

Call XORDELBEG(START)

Translating XORDELBEG(START):

XORDELBEG(START):

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete."

Return and EXIT.

[End of If structure]

2. [INITIALIZE TEMPORARY POINTER]

Set TEMPPTR := START $\left(\begin{array}{l} \text{Store the current head to free} \\ \text{its memory later} \end{array} \right)$

3. [DECODE THE SECOND NODE]

Set NEXTTEMPPTR := NULL $\oplus$ NEXTXORPTR[START] $\left(\begin{array}{l} \text{Calculate the} \\ \text{address of the node} \\ \text{that will become} \\ \text{the new head} \end{array} \right)$

4. [UPDATE NEW HEAD'S LINKAGE]

If NEXTTEMPPTR $\neq$ NULL, then: $\left(\begin{array}{l} \text{If the list has more} \\ \text{than one node,} \\ \text{we must update} \\ \text{the successor's NPX} \end{array} \right)$

a. Set NEXTNEXTTEMPPTR :
 = TEMPPTR

$\oplus$ NEXTXORPTR[NEXTTEMPPTR] $\left(\begin{array}{l} \text{Decode the node following} \\ \text{the second node using} \\ \text{the current head} \\ \text{as the 'previous'} \\ \text{reference} \end{array} \right)$

b. Set NEXTXORPTR[NEXTTEMPPTR]:

= NULL $\oplus$ NEXTNEXTTEMPPTR $\left(\begin{array}{l} \text{Update the new head's NPX} \\ \text{to indicate its} \\ \text{previous neighbor} \\ \text{is now NULL} \end{array} \right)$

[End of If structure]

5. [UPDATE GLOBAL START]

Set START := NEXTTEMPPTR $\left(\begin{array}{l} \text{The second node officially} \\ \text{becomes the} \\ \text{first node} \\ \text{of the list} \end{array} \right)$

6. [FREE MEMORY AND PRINT]

Write: Deleted ,INFO[TEMPPTR], from the beginning.

FREE(TEMPPTR) $\left(\begin{array}{l} \text{Return the memory of the} \\ \text{deleted node to the system} \end{array} \right)$

7. EXIT

Return and **EXIT**.
 [End of If structure]

3. [INITIALIZE TRAVERSAL]

Set CURRPTR := START
 Set PREVPTR := NULL
 Set COUNT := 1

4. [TRAVERSE TO POSITION POS]

Repeat while CURRPTR $\neq$ NULL AND COUNT < POS:

- a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]*
- b. Set PREVPTR := CURRPTR*
- c. Set CURRPTR := NEXTTEMPPTR*
- d. Set COUNT := COUNT + 1*

[End of Loop]

5. [CHECK BOUNDS]

If CURRPTR = NULL, then:

Write: "Position out of bounds!"

Return and EXIT.

[End of If structure]

6. [DECODE NEIGHBORS]

Set NEXTTEMPPTR :

$$= \text{PREVPTR} \oplus \text{NEXTXORPTR}[\text{CURRPTR}] \left(\begin{array}{c} \text{This decodes the node} \\ \text{following the} \\ \text{one to be deleted} \end{array} \right)$$

7. [UPDATE PREVIOUS NODE]

a. Set PREVPREVTEMPPTR := NEXTXORPTR[PREVPTR] $\oplus$ CURRPTR

b. Set NEXTXORPTR[PREVPTR]:

$$= \text{PREVPREVTEMPPTR}$$

$$\oplus \text{NEXTTEMPPTR} \left(\begin{array}{c} \text{Re - link the previous node to} \\ \text{bypass CURRPTR} \\ \text{and point to NEXTTEMPPTR} \end{array} \right)$$

8. [UPDATE NEXT NODE IF IT EXISTS]

If NEXTTEMPPTR $\neq$ NULL, then:

a. Set NEXTNEXTTEMPPTR := CURRPTR $\oplus$ NEXTXORPTR[NEXTTEMPPTR]

b. Set NEXTXORPTR[NEXTTEMPPTR]

$$:= \text{PREVPTR}$$

$$\oplus \text{NEXTNEXTTEMPPTR} \left(\begin{array}{c} \text{Re - link the successor node to bypass} \\ \text{CURRPTR and point back to PREVPTTR} \end{array} \right)$$

[End of If structure]

9. [FREE MEMORY AND PRINT]

Write: "Deleted ", INFO[CURRPTR], " from position ", POS

FREE(CURRPTR)

10. EXIT
